



C++ Programming Language

Presented By

Usha Rani Sargurunathan

STG Group

Variable Parameter List (Variadic Functions)

Variable Parameter List (Variadic Functions)

- **Definition:** Functions that can accept a variable number of arguments.
- In C++, this is typically handled using variadic templates or, for C-style functions, using the `<cstdarg>` header.
- **C-style Variadic Functions (using `stdarg.h`):**
 - Uses macros like `va_list`, `va_start`, `va_arg`, `va_end`.
 - Type safety is not enforced by the compiler, leading to potential runtime errors.
- **C++11 Variadic Templates:**
 - A type-safe and more powerful way to handle variable arguments.
 - Uses template parameter packs.
 - Preferred approach in modern C++.
- **Purpose:** To create highly flexible functions that can operate on an arbitrary number of inputs.

Variable Parameter List (Variadic Functions)

Core Components: Inherited from C, this method uses a special syntax with an ellipsis (...) in the function signature. It relies on macros defined in the `<stdarg.h>` library to extract arguments at runtime.

- **The Ellipsis (...):** Placed at the end of the parameter list to signify that the function accepts extra, unnamed arguments.
- **va_list:** A special type/pointer used to hold and keep track of the variable arguments.
- **va_start(args_ptr, last_fixed_param):** A macro that initializes the va_list pointer to point to the first variable argument.
- **va_arg(args_ptr, type):** A macro that extracts the next argument from the list and advances the pointer. You must explicitly supply the expected data type.
- **va_end(args_ptr):** A macro used to clean up the va_list resources before exiting the function.

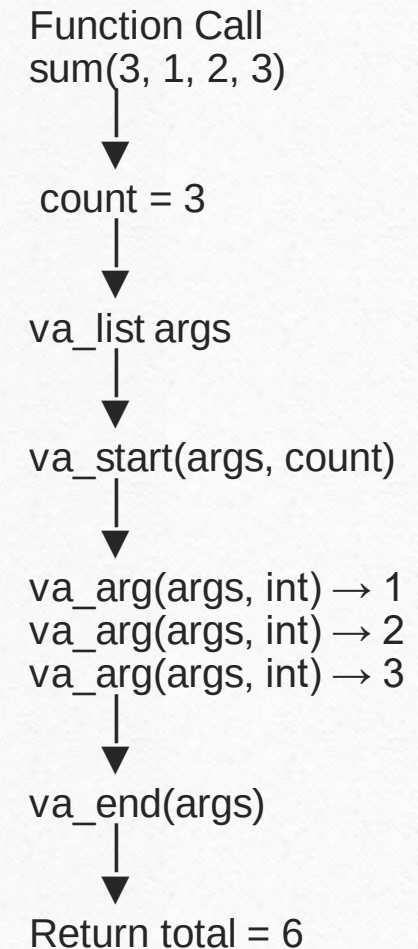
Example – Sum of 2 Integers

```
# include<iostream>
using namespace std;
void sum(int, int);
int main() {
    sum(3,5);
    cout<<endl;
    return 0;
}
void sum(int a, int b) {
    cout << a + b << endl;
}
```

Example - Variable Parameter List (Sum)

```
#include<iostream>
#include<cstdarg>
using namespace std;
int sum(int count, ...) {
    va_list args;
    va_start(args, count);
    int sum = 0;
    for (int i = 0; i < count; ++i) {
        sum += va_arg(args, int);
    }
    va_end(args);
    return sum; }
```

```
int main() {
    int result = sum(3, 1, 2, 3);
    cout << "The sum is: " <<
    result << endl;
    return 0;
}
```



Example - Variable Parameter List(Average)

```
#include <iostream>
#include <cstdarg>

double average(int num_args, ...) {
    va_list ap;
    double sum = 0.0;
    va_start(ap, num_args); // Initialize argument list
    for (int i = 0; i < num_args; ++i) {
        sum += va_arg(ap, int); // Access arguments
    }
    va_end(ap); // Clean up
    return sum / num_args; }

int main() {
    std::cout << "Average of 1, 2, 3: " <<
    average(3, 1, 2, 3) << std::endl;

    std::cout << "Average of 10, 20, 30, 40:
    " << average(4, 10, 20, 30, 40) <<
    std::endl;

    return 0;
}
```

Example - Variable Parameter List(Max)

```
#include <iostream>
#include <cstdarg>
using namespace std;
int maximum(int count, ...)
{
    va_list args;
    va_start(args, count);
    int max = va_arg(args, int);
    for (int i = 1; i < count; i++)
    {
        int value = va_arg(args, int);
        if (value > max)
            max = value;
    }
    va_end(args);
    return max;
}
```

```
int main()
{
    cout << "Maximum: " << maximum(4, 15,
    8, 25, 10);

    return 0;
}
```

Task

Create a variadic function that accepts a variable number of integers and displays the number of even and odd values.

Example: analyzeNumbers(6, 10, 15, 20, 25, 30, 35)

Expected Output:

Even numbers = 3

Odd numbers = 3

Solution

```
#include <iostream>
#include <cstdarg>
using namespace std;
void analyzeNumbers(int count, ...)
{
    va_list args;
    va_start(args, count);
    int evenCount = 0;
    int oddCount = 0;
    for (int i = 0; i < count; i++)
    {
        int number = va_arg(args, int);
        if (number % 2 == 0)
        {
            evenCount++;
        }
        else
        {
            oddCount++;
        }
    }
}
```

```
va_end(args);
    cout << "Even numbers = " << evenCount
    << endl;
    cout << "Odd numbers = " << oddCount
    << endl;
}
int main()
{
    analyzeNumbers(6, 10, 15, 20, 25, 30,
    35);
    return 0;
}
```

Task

Create a variadic function that accepts a variable number of integers and counts Positive, Negative and Zero Values.

Example: `analyze(7, 10, -5, 0, 25, -8, 0, 15)`

Expected Output:

Positive = 3

Negative = 2

Zero = 2

Constructors and Memory Allocation

Managing the Heap (Dynamic Memory Allocation)

Heap (Free Store): A region of memory where objects can be allocated and deallocated dynamically at runtime.

new operator: Used to allocate memory on the heap for an object and call its constructor. Returns a pointer to the newly allocated memory.

- Example: `MyClass *objPtr = new MyClass();`

delete operator: Used to deallocate memory on the heap that was previously allocated with `new`, and calls the object's destructor.

- Example: `delete objPtr;`

Importance: Crucial for objects whose lifetime is not known at compile time or whose size varies.

Caution: Failure to `delete` allocated memory leads to memory leaks.

Dynamic Memory Management: new , malloc , calloc

- **new (C++ Operator):**
 - **Purpose:** Dynamically allocates memory for an object (or array of objects) on the heap and **calls the constructor(s)**.
 - **Return Type:** Returns a type-safe pointer to the allocated object.
 - **Error Handling:** Throws `std::bad_alloc` on allocation failure (or returns `nullptr` with `std::nothrow`).
 - **Usage:** `MyClass *obj = new MyClass();` or `int *arr = new int[10];`
- **malloc and calloc (C Function, from <cstdlib>):**
 - **Purpose:** Dynamically allocates a block of raw memory of a specified size (in bytes) on the heap.
 - **Usage(malloc):** `MyClass *obj = (MyClass*)malloc(sizeof(MyClass));`
 - **Usage(calloc):** `MyClass *obj = (MyClass*)calloc(1, sizeof(MyClass));`
- **Key Difference:** `new` understands objects and calls constructors/destructors; `malloc/calloc` only deal with raw memory. `new` is generally preferred in C++.

Dynamic Memory Management::new - Example

```
#include <iostream>
using namespace std;
int main()
{
    int *p = NULL;
    p = new int(10);
    int *p2 = new int[20];
    cout<<*p;
    delete p;
    delete[] p2;
    return 0;
}
```

```
#include <iostream>
using namespace std;
class Room{
    int length;
    int breadth;
public:
    Room(int l, int b) {
        length = l;
        breadth = b;
        cout<< "Length: "<<length<<endl;
        cout<< "Breadth: "<<breadth<<endl;
    };
    int main() {
        Room *p = new Room(10, 20);
        return 0; }
}
```

Task: Dynamic Student Grade Tracker

Write a program that asks an instructor how many students are in their class.

Dynamically allocate an array of float on the Heap to store their final grades.

Prompt the instructor to enter each grade, print out all grades, and then calculate and display the highest grade.

Finally, deallocate the memory properly to prevent leaks.

Solution

```
#include <iostream>

int main() {
    int N = 0;
    // Prompt user for the number of students
    std::cout << "Enter the number of students: ";
    std::cin >> N;
    // Input validation
    if (N <= 0) {
        std::cout << "Invalid number of students. Program exiting.\n";
        return 1; }
    // Allocate memory on the Heap for an array of float of size N
    float *grades = new float[N];
    // Input grades using a loop
    std::cout << "\nEnter the final grades for " << N << "
students:\n";
    for (int i = 0; i < N; ++i) {
        std::cout << "Grade for Student " << (i + 1) << ": ";
        std::cin >> grades[i];
    }

    // Display all entered grades
    std::cout << "\n--- Student Grades ---\n";
    for (int i = 0; i < N; ++i) {
        std::cout << "Student " << (i + 1) << ": " <<
grades[i] << "\n";
    }
    // Find and display the maximum grade
    float maxGrade = grades[0];
    for (int i = 1; i < N; ++i) {
        if (grades[i] > maxGrade) {
            maxGrade = grades[i];
        }
    }
    std::cout << "\nHighest Grade in Class: " <<
maxGrade << "\n";
    delete[] grades;
    return 0;
}
```

Types of Constructors

Copy Constructor

- **Definition:** A special constructor that initializes a **new object** as a copy of an **existing object** of the same class.
- **Signature:** `ClassName(const ClassName& other);`
- **When called:**
 - When an object is initialized by another object of the same class (`MyClass obj2 = obj1;` or `MyClass obj2(obj1);`).
 - When an object is passed by value to a function.
 - When an object is returned by value from a function.

Relevance to Copy Constructor

The copy constructor often takes a `const` reference to an object as its parameter.

- `ClassName(const ClassName& other);`

Why `const` reference?

- **Efficiency:** Avoids unnecessary copying of the object being copied (which would recursively call the copy constructor).
- **Safety:** Ensures that the **original object is not modified** during the copy operation.

This reinforces the use of references for efficient and safe object handling in C++.

```
#include <iostream>
using namespace std;
class Complex {
private:
    int real;
    int imag;
public:
    Complex (int r, int i) {
        real = r;
        imag = i; }
    Complex (const Complex &c) {
        real = c.real;
        imag = c.imag; }}

void display()
{
    cout << real << "+" << imag << "i"
    << endl;
}

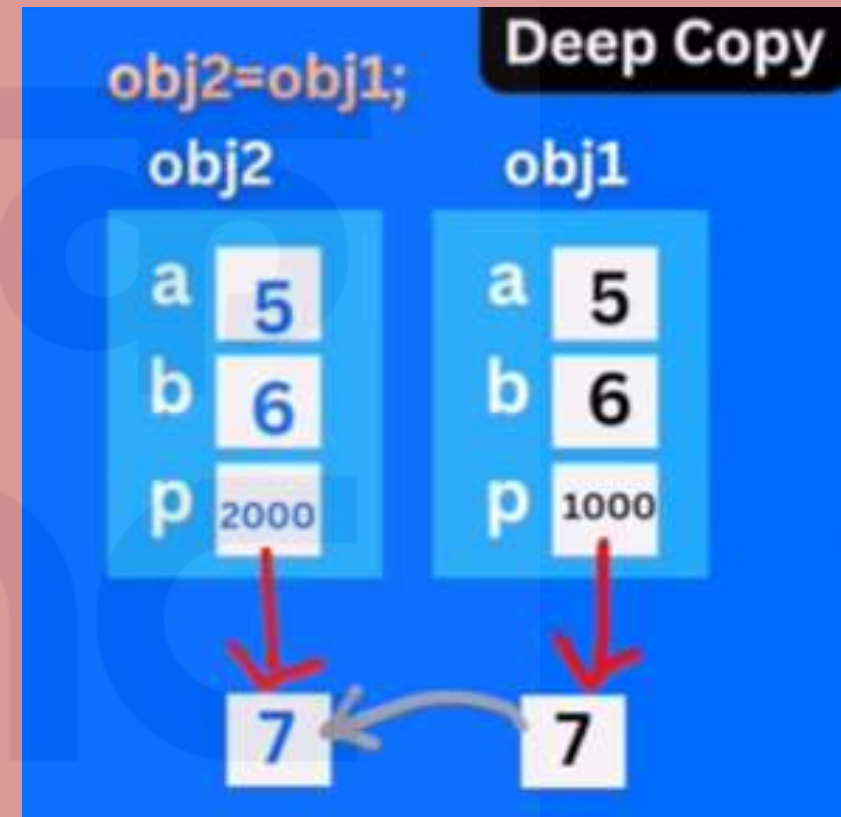
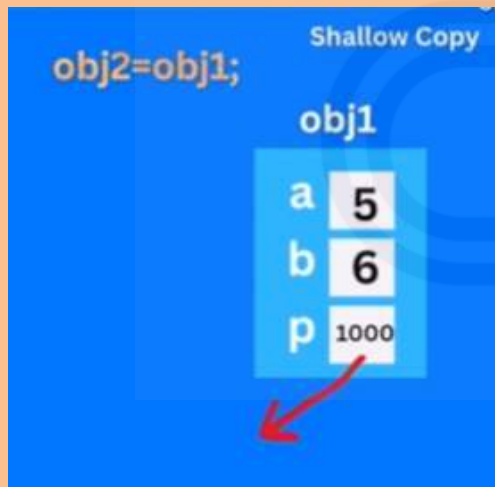
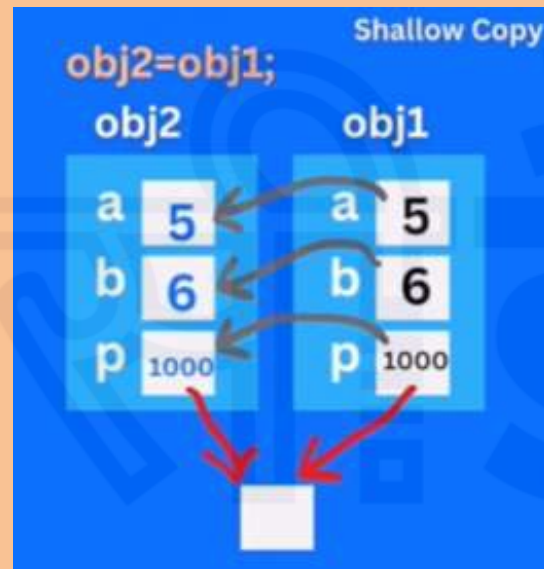
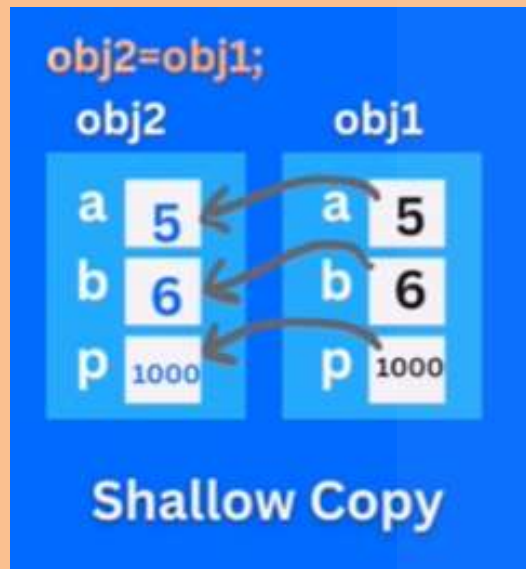
};

int main()
{
    Complex c1(3, 4);
    Complex c2 = c1; // Copy constructor is
called here
    c1.display();
    c2.display();
    return 0;
}
```

Shallow Copy v/s Deep Copy

- **Shallow Copy:**
 - **Mechanism:** A bit-wise **copy of the object's data members**.
 - **Behavior:** If an object contains pointers to dynamically allocated memory, only the pointer itself is copied, not the data it points to. **Both original and copied objects will point to the *same* dynamically allocated memory.**
 - **Problem:** Leads to "double-free" issues and unexpected modifications if one object deallocates or modifies the shared memory.
 - **When it occurs:** By default, with compiler-generated copy constructors and assignment operators.
- **Deep Copy:**
 - **Mechanism:** Creates a **new, independent copy of all data**, including any dynamically allocated memory pointed to by members.
 - **Behavior:** If an object contains pointers, new memory is allocated for the pointed-to data, and the content is copied. Original and copied objects have their own independent copies of all resources.
 - **Solution:** Requires a user-defined copy constructor and copy assignment operator to explicitly handle the allocation and copying of dynamic resources.
- **Importance:** Crucial for classes managing dynamic memory to prevent resource leaks and corruption.

Shallow Copy v/s Deep Copy



Shallow Copy

```
#include <iostream>
using namespace std;
class Test
{
public:
    int a;
    int *p;
    Test(int x)
    {
        a = x;
        p = new int[a];
    }
    Test(const Test &t)
    {
        a = t.a;
        p = t.p; // Shallow copy
    }
};
```

```
int main()
{
    Test t(5);
    Test t1(t);
    cout<< "t.a = " << t.a << endl;
    cout<< "t.p = " << t.p << endl;
    cout<< "t1.a = " << t1.a << endl;
    cout<< "t1.p = " << t1.p << endl;
    return 0;
}
```

Deep Copy

```
#include <iostream>
using namespace std;
class Test {
public:
    int a;
    int *p;
    Test(int x)
    {
        a = x;
        p = new int[a];
    }
    Test(const Test &t)
    {
        a = t.a;
        p = new int[a]; // Deep copy
        for(int i = 0; i < a; i++)
        {
            p[i] = t.p[i];
        }
    }
};
```

```
int main()
{
    Test t(5);
    Test t1(t);
    cout<< "t.a = " << t.a << endl;
    cout<< "t.p = " << t.p << endl;
    cout<< "t1.a = " << t1.a << endl;
    cout<< "t1.p = " << t1.p << endl;
    return 0;
}
```

Task – Shallow Copy

Write a C++ program to create a **class named Student** with the following members:

int rollNo

int *marks (allocate memory dynamically)

The class should have:

A parameterized constructor to initialize rollNo and marks.

A display() function to print the roll number, marks, and the memory address of marks.

A setMarks() function to modify the marks.

In the main() function:

Create one object s1.

Create another object s2 by copying s1 using the default copy constructor.

Display both objects.

Modify the marks of s2.

Display both objects again.

Task – Deep Copy

Modify the previous program by implementing a copy constructor that performs a deep copy.

Requirements

Allocate separate memory for the copied object.

Copy the value stored in marks instead of copying the pointer.

Display the addresses of both pointers.

Verify that changing one object's marks does not affect the other.

Constructors and Initialization

Initialization List

- **Definition:** A mechanism in a constructor to initialize data members (especially `const` members, references, and member objects without default constructors) *before the constructor's body executes*.
- **Syntax:** `ConstructorName(parameters) : member1(arg1), member2(arg2, arg3)`
`{ /* constructor body */ }`
- **Why use it?**
 - **Mandatory for:** `const` data members, reference data members, and member objects that **do not have a default constructor**.
 - **Efficiency:** Direct initialization is generally more efficient than assignment within the constructor body, especially for complex objects (avoids default construction + assignment).
 - **Correctness:** Ensures members are initialized, not just assigned.

Before using Initialization List

```
#include <iostream>
using namespace std;
class Base {
public:
    int x;
    Base(int a) {
        x = a;
    } };
int main() {
    Base b(10);
    cout << b.x << endl;
    return 0; }
```

Using Initialization List

```
#include <iostream>
using namespace std;
class Base {
public:
    int x;
    Base(int a): x{a} {}
};
int main() {
    Base b(10);
    cout << b.x << endl;
    return 0;
}
```

Example - Initialization List

```
class MyClass {  
    const int ID;  
    std::string name;  
public:  
    MyClass(int id, const std::string& n) : ID(id), name(n) {  
        // ID and name are initialized here, not assigned  
    }  
};
```

Const & Reference

Why constructor initialization lists are **mandatory** for **const** and **reference** members in C++:

- By the time the code reaches the opening brace { of the constructor body, **all member variables have already been created (allocated memory)**.
- **const** variables cannot be modified after they are created.
- **Reference variables (&)** must be bound to an object at the exact moment they are created.
- Therefore, you cannot assign values to them inside the body; they *must* be initialized beforehand using the initialization list.

Example – Const & Reference

```
#include <iostream>
class Example {
private:
    const int max_value;    // Constant member
    int &original_ref;      // Reference member
public:
    // CORRECT: Initializing directly before creation
    Example(int max, int& external_var) : max_value(max),
    original_ref(external_var) {
        // Constructor body
        // Both max_value and original_ref are already safely
        set here. }
    /* WRONG (Will not compile):
    Example(int max, int& external_var) {
        max_value = max;      // Error: Cannot assign to a
        const variable
        original_ref = external_var; // Error: Reference not
        initialized at creation } */
```

```
void display() const {
    std::cout << "Const Max: " <<
    max_value << ", Ref Value: " <<
    original_ref << std::endl;
}
};

int main() {
    int target = 42;
    Example ex(100, target);
    ex.display(); }
```

Order of Initialization

- **Rule:** Data members of a class are initialized in the **order they are declared in the class definition**, *not* in the order they appear in the constructor's initializer list.
- **Importance:** If one member's initialization depends on another, ensure the dependency is declared first in the class. Otherwise, you might use an uninitialized value.
- **Order of Construction (Full Object):**
 1. Base class constructors (in order of inheritance).
 2. Member objects' constructors (in order of declaration).
 3. The constructor body of the current class.

Example:

```
class OrderExample {  
    int b;  
    int a; // 'a' is declared after 'b'  
public:  
    // Initializer list order doesn't change  
    declaration order  
    OrderExample() : a(10), b(a) { // 'b' initialized  
        before 'a' is truly initialized  
        // std::cout << "a: " << a << ", b: " << b  
        << std::endl;  
    }  
};
```

Task

Write a C++ program to create a class Number having:

int value

int &ref

Requirements:

Initialize the reference member using an initialization list.

Display both value and ref.

Modify value and verify that ref also changes.

Task

Create a class Rectangle with:

int length

int breadth

Requirements:

Initialize both members using an initialization list.

Implement a function to calculate the area.

Display the dimensions and area.

Strings in C++

String - Definition & Concept

- In C++, memory allocation for text can be static (fixed array) or dynamic (resizable string objects).
- Dual Approaches in C++: C++ offers two distinct mechanisms for handling text data based on historical evolution:
 - C-Style Strings: Low-level character arrays inherited directly from the C language.
 - `std::string` Class: High-level, safe, feature-rich object from the C++ Standard Library.

```
char name1[10] = "Usha Rani";
```

```
std::cout << "Hello, " << name1 << "!" << std::endl; or
```

```
std::string name2 = "Usha Rani";
```

```
std::cout << "Hello, " << name2 << "!" << std::endl;
```

String - Definition & Concept

std::string Class (Modern)

Standard library wrapper offering automatic dynamic resizing and memory safety.

```
#include <string>  
std::string str = "Modern C++";
```

Highly recommended for modern C++ development; handles memory automatically.

Declare and Initialize strings

```
#include <iostream>
#include <string>
int main() {
    std::string s1 = "Hello"; // Direct & Copy initialization
    std::string s2("World");
    std::string s3(5, 'A');
    return 0;
}
```

Get String Input From User - cin

Example:

```
std::string name;  
std::cout << "Enter your name: ";  
std::cin >> name;  
std::cout << "Hello, " << name << "!" << std::endl;
```

Get String Input From User – getline()

```
std::cout << "Enter your name: ";
```

```
std::getline(std::cin, name);
```

```
std::cout << "Hello, " << name << "!" << std::endl;
```

Best Practices

- **Prefer `std::string`:** Always use `std::string` over char arrays in modern C++ to prevent buffer overflows and memory corruption.
- **Use `getline()` for sentences:** Rely on `std::getline(std::cin, str)` whenever expecting spaces in user input.

Operations: Copy, Concatenate & Length

Operation	Modern std::string	Legacy C-Style (<cstring>)
Find Length	<code>str.length()</code> or <code>str.size()</code>	<code>strlen(str)</code>
Copy String	<code>dest = src;</code> or <code>dest.assign(src);</code>	<code>strcpy(dest, src);</code>
Concatenate	<code>str1 + str2</code> or <code>str1.append(str2);</code>	<code>strcat(dest, src);</code>
Compare	<code>str1 == str2</code> or <code>str1.compare(str2)</code>	<code>strcmp(str1, str2) == 0</code>

Example

```
// Example of std::string concatenation & length  
std::string a = "Hello ", b = "World";  
std::string c = a + b; // "Hello World"  
size_t len = c.length(); // 11
```

Example

```
#include <iostream>
#include <string>
int main() {
    // 1. Creation & Copying
    std::string original = "Apple";
    std::string copy = original; //copy

    // 2. Finding Length
    std::cout << "Original String: " << original << "\n";
    std::cout << "Length: " << original.length() << " characters\n\n";

    // 3. Concatenation (Joining strings)
    std::string first = "Hello, ";
    std::string second = "World!";
    std::string fullMessage = first + second; // + operator
    std::cout << "Concatenated String: " << fullMessage << "\n\n";
```

```
    // 4. Comparison
    std::string str1 = "Cat";
    std::string str2 = "Dog";

    if (str1 == str2) {
        std::cout << "Strings are equal.\n";
    } else {
        std::cout << "\"" << str1 << "\" and \""
        << str2 << "\" are NOT equal.\n";
    }

    return 0;
}
```

Task

- Write a C++ program that asks the user to enter a single word, then complete the following steps:
- **Copy:** Copy the input word into a new string variable named username.
- **Concatenate:** Create a new string named fullTag by joining username with the suffix "_2026".
- **Length:** Calculate and display the total number of characters in fullTag.
- **Compare:** Check if fullTag is equal to "admin_2026".
 - If equal, display: "Access: Admin access granted!"
 - Otherwise, display: "Access: Regular user account created."

Destructor

Destructor

- **Definition:** A special member function that is automatically called when **an object is destroyed**.
- **Purpose:** To perform necessary **cleanup operations** before the object's memory is reclaimed. This primarily involves **releasing resources acquired by the object during its lifetime**.
- **Key Responsibilities:**
 - Deallocating dynamically allocated memory using `delete` or `delete[]`.
 - Closing open files or network connections.
 - Releasing mutexes or other synchronization primitives.
 - Any other resource cleanup specific to the class.
- **Signature:** `~ClassName();` (no return type, no parameters).
- **Execution:**
 - For stack-allocated objects: when they go out of scope.
 - For heap-allocated objects: when `delete` is called on their pointer.
 - For static/global objects: at program termination.

delete vs free

- **delete (C++ Operator):**
 - **Purpose:** Deallocates memory previously allocated by `new` (or `new[]`).
 - **Behavior:** For objects, it first calls the object's **destructor** and then deallocates the memory. For arrays, it calls destructors for all elements and then deallocates the array memory.
 - **Type-aware:** Understands object types and their destructors.
 - **Usage:** `delete ptr;` for single objects; `delete[] arrPtr;` for arrays.
- **free (C Function, from `<cstdlib>`):**
 - **Purpose:** Deallocates a block of raw memory previously allocated by `malloc`, `calloc`, or `realloc`.
 - **Behavior:** Only deallocates the raw memory. It **does not call destructors** for C++ objects.
 - **Type-unaware:** Deals only with raw memory blocks.
 - **Usage:** `free(ptr);`
- **Key Difference:** `delete` is for C++ objects (calls destructors); `free` is for raw memory (does not call destructors). Mismatching them (e.g., `free` on `new`'d memory) leads to undefined behavior.

Compiler Synthesized Destructor

- **Definition:** If you don't explicitly define a destructor for a class, the C++ compiler automatically generates (synthesizes) a **default destructor**.
- **Characteristics:**
 - It's a **public, no-argument destructor**.
 - It performs cleanup for non-static data members: it calls the destructors for any member objects and does nothing for built-in types.
 - It **does not deallocate dynamically allocated memory** (e.g., pointed to by raw pointers within the class).
- **When it's generated:** If no user-defined destructor is provided.
- **Problem with dynamic memory:** If your class manages dynamic memory (e.g., has `char*` or `int*` members allocated with `new`), the **compiler-synthesized destructor will *not delete* that memory**, leading to memory leaks. This is why you often need to define your own destructor for such classes.

Example - Destructor

```
#include<iostream>
using namespace std;
class demo
{
    int *p;
public:
    demo()
    {
        p=new int[10];
        cout<<"constructor of demo"<<endl;
    }

    ~demo()
    {
        delete[] p;
        cout<<"destructor of demo"<<endl;
    } };
    void fun()
    {
        demo *p=new demo();
        delete p; }
    int main()
    {
        fun(); }
```